

HPC Benchmark Analysis Report

Not So Slow Slugs Team 1

April 13, 2026

1 Which images (“nodes”) are broken?

Table 1: Summary of Broken Images

Image	Issue
image_2	blas cannot be found
image_3	multinode MPI cannot run
image_6	MPI is just broken

The rest are valid/working.

2 Detailed results for each test + node combination

For each test, we averaged the results of multiple identical runs (the number is indicated by count). “None” means there were no valid runs.

2.1 HPL 1 Node run

Table 2: HPL 1 Node Results

Image	Count	Time	Gflops
image_1	34	1065.389118	1514.15
image_2	7	None	None
image_3	5	1062.716	1517.56
image_4	29	1064.41931	1515.568966
image_5	30	1067.636667	1511.013333
image_6	7	None	None
image_7	25	1067.5152	1511.144
image_8	15	1063.376667	1517.006667

The HPL 1 Node results are about what we would expect given the issues noted above. Most images (except 2 and 6) are able to complete a run with a result of about 1515 Gflops. Without BLAS, HPL is unable to run on image 2 and on image 6 HPL cannot run with the broken MPI. Image 3 is able to run because it is on a single node and MPI has multinode issues.

2.2 HPL 2 Node runs

Table 3: HPL 2 Node Results

Image	Count	Time	Gflops
image_1	8	519.0425	2378.9125
image_2	8	None	None
image_3	8	None	None
image_4	8	518.83375	2379.9625
image_5	8	516.12375	2392.75
image_6	8	None	None
image_7	8	511.14125	2416.1375
image_8	8	512.3425	2410.1875

When running HPL on 2 nodes we see consistent performance of around 2400 Gflops on 5 of the images. Images 2 and 6 fail for the same reasons as in the single node run, with 2 missing BLAS and 6 having a broken install of MPI. Image 3 fails for the multinode run, which we concluded was an mpi issue allowing HPL to run within node but not across nodes.

2.3 LAMMPS reax

Table 4: LAMMPS Results

Image	Count	Volume	Initial Energy/Volume	Maximum Energy Drift	Valid
image_1	7	322443.3661	-9.082344398	0.0003504491389	1
image_2	7	322443.3661	-9.082344398	0.0003504491389	1
image_3	7	322443.3661	-9.082344398	0.0003504491389	1
image_4	7	322443.3661	-9.082344398	0.0003504491389	1
image_5	7	322443.3661	-9.082344398	0.0003504491389	1
image_6	0	None	None	None	None
image_7	7	322443.3661	-9.082344398	0.0003504491389	1
image_8	7	322443.3661	-9.082344398	0.0003504491389	1

The energy drift was calculated by comparing every 2 pairs of datapoints and checking if the rate of change is more than expected: 5e-4. This was done through python in our collectdata.py file. These results are reasonable and identical since LAMMPS is deterministic. LAMMPS fails on node 6 because of the MPI issue.

2.4 OSU BW (Bandwidth)

The last 4 columns describe the size of the messages, and the data is in MB/s. There are many more values for other sizes, but we’re showing the smallest/largest 2.

Table 5: OSU Bandwidth Results (MB/s)

Image	Count	1	2	2097152	4194304
image_1	1861	0.2915690489	0.5936808168	5586.989764	5633.217136
image_2	1653	0.3066727163	0.6267211131	5903.018851	5949.049613
image_3	19	None	None	None	None
image_4	1509	0.3081179589	0.6279522863	5871.766673	5920.714009
image_5	1423	0.3093745608	0.63037948	5929.413591	5979.422677
image_6	9	None	None	None	None
image_7	1548	0.3075839793	0.6262209302	5878.078947	5927.684974
image_8	1823	0.3060175535	0.6238288535	5904.93616	5949.036648

As expected, increasing the message size increase bandwidth until it caps out at the network limit.

2.5 OSU Latency

The last 4 columns describe the size of the messages, and the data is in μ s. There are many more values for other sizes, but we’re showing the smallest/largest 2.

Table 6: OSU Latency Results (μ s)

Image	Count	1	2	2097152	4194304
image_1	1861	10.66396024	10.65333154	356.9498818	683.0992907
image_2	1653	11.27362371	11.26298851	380.1029159	728.442559
image_3	10	None	None	None	None
image_4	1509	11.28836978	11.28231942	378.3135321	725.2237972
image_5	1423	11.26319044	11.25542516	381.7939494	731.4990654
image_6	9	None	None	None	None
image_7	1548	11.2676938	11.2574677	377.9893863	725.2141925
image_8	1822	11.29152031	11.28030187	379.5784962	727.2368551

As expected, increasing the message size increase latency; it is roughly linear.

3 Anomalous compute nodes

There are few nodes that are unavailable (not broken, but unusable for us). We checked how they were doing using “scontrol show node”:

- [odo01–odo04] — used for Kubernetes (reserved)
- odo09 — been under maintenance for some time (drained/down)
- odo10 — also has been under maintenance, but more recently (down)
- A few more nodes were changed into the “plnd” state, but we don’t know much about them

Technically speaking, since we can’t use them, they’re very bad.

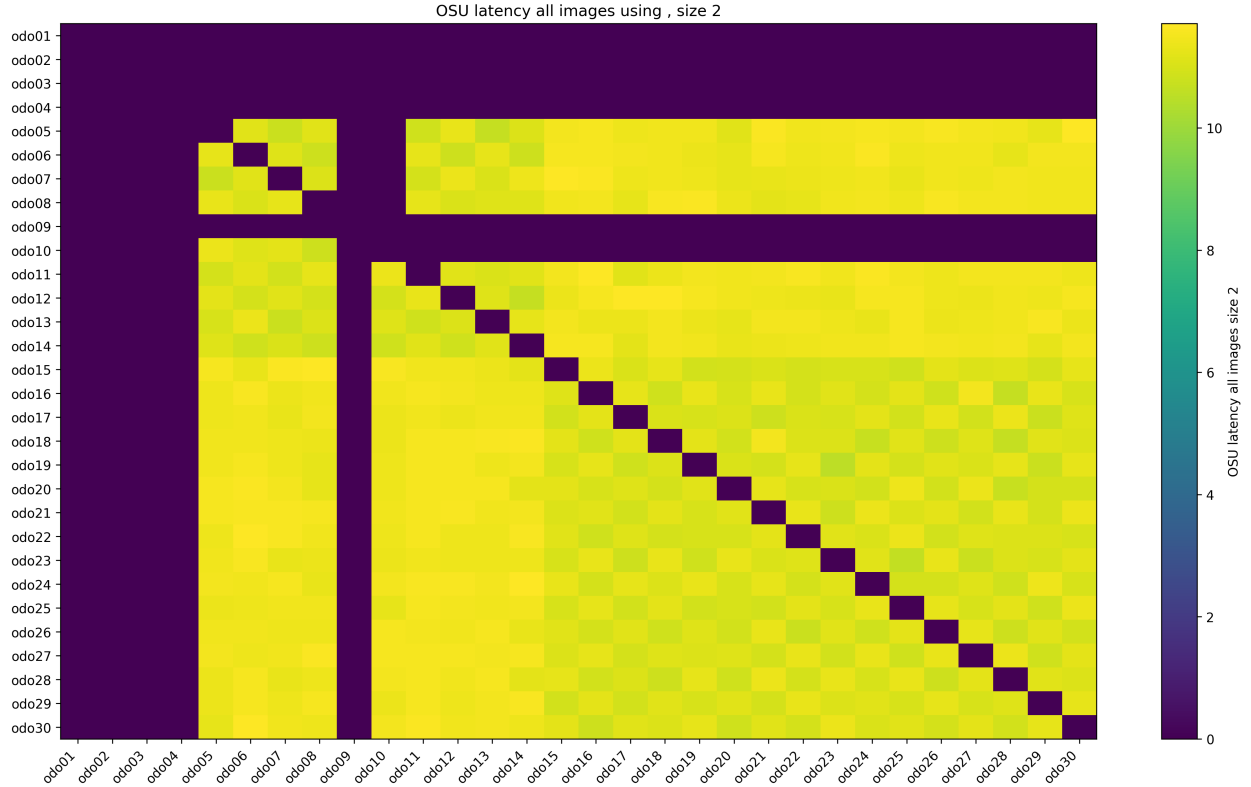


Figure 1: Latency heatmap between nodes, size 2

Since we collected a lot of data between all possible nodes (missing entries are due to node unavailability), we can see a very minor difference in the latency between nodes. Communication between the upper half and bottom half of numbered nodes seems to be slower than to their own half. The nodes probably are grouped in by lower numbers (01 - 14), and upper ones (15 - 30), likely physically separated by some marginal amount.

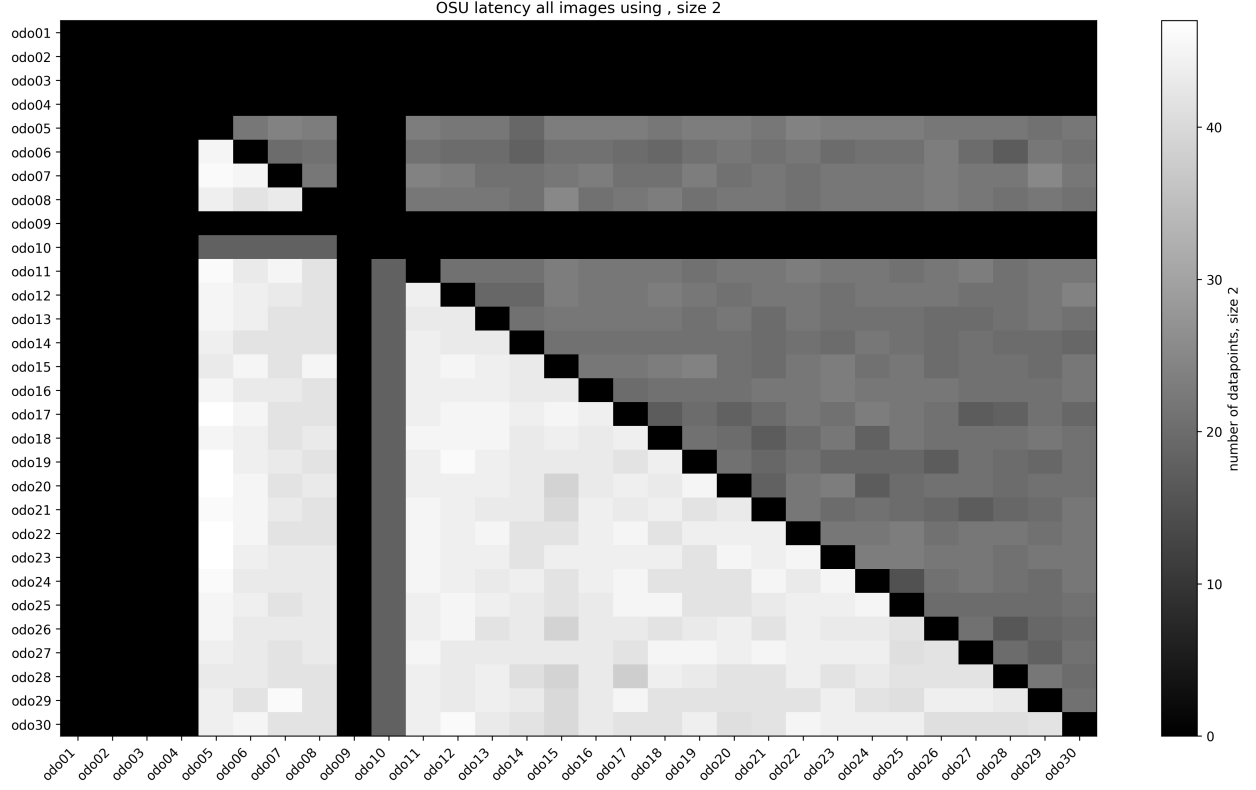


Figure 2: Number of tests per between nodes

We verify that this difference is unlikely due to bias in data sampling. However, this minor change doesn't scale up, and bandwidth seems to be independent of nodes (expected, as the communication method should be identical, only distance may change).

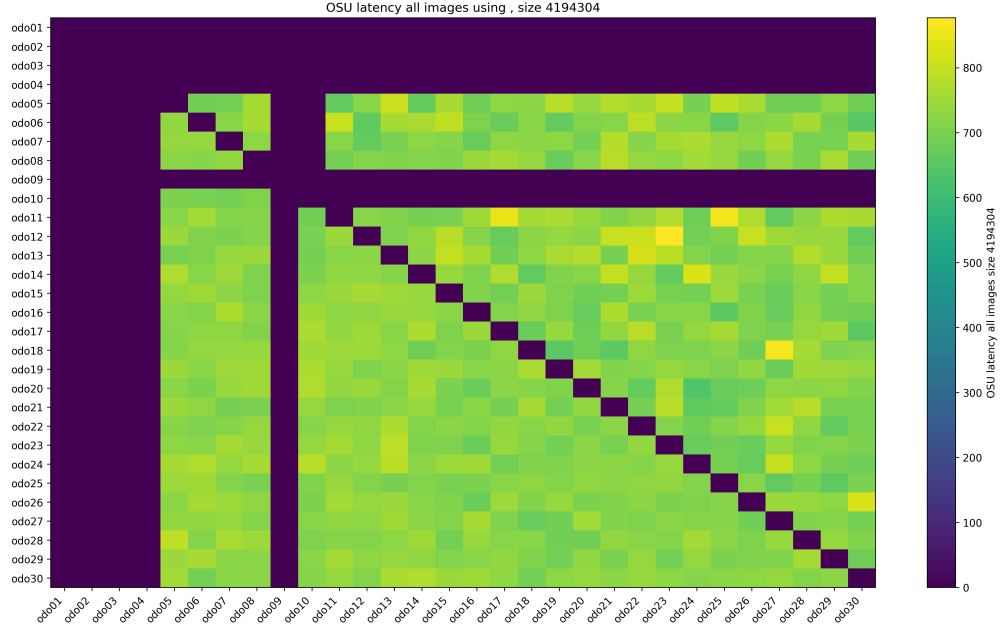


Figure 3: Latency using size 4194304

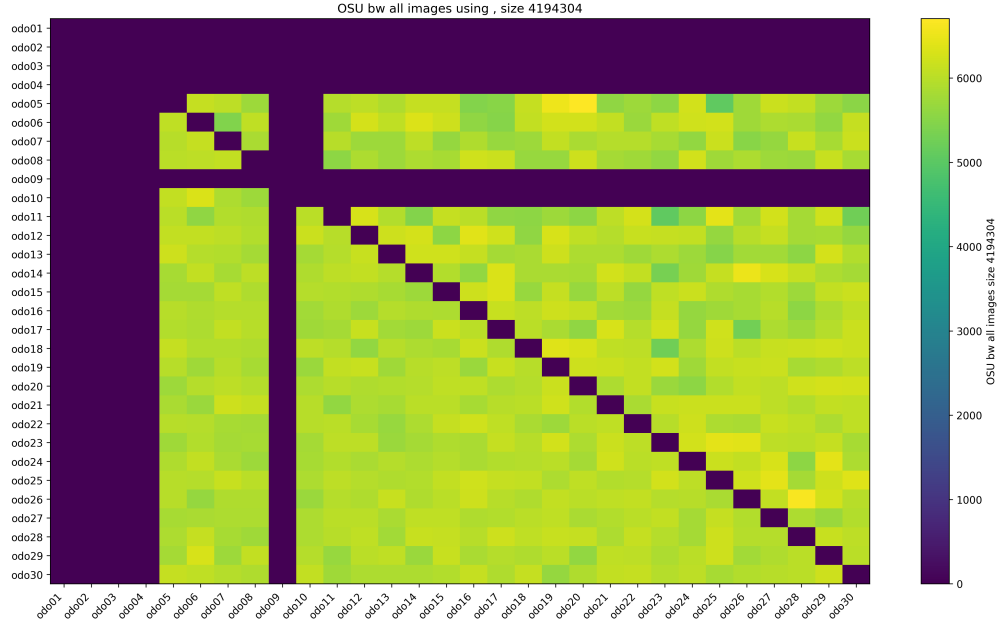


Figure 4: Bandwidth using size 4194304

We also compare the HPL GFLOPs across nodes.

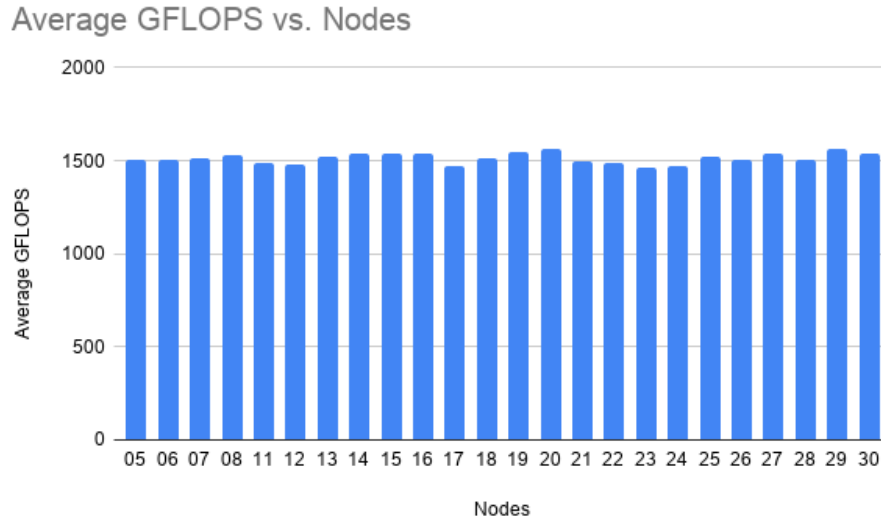


Figure 5: GFLOPs across different nodes

We conclude that all nodes except for the ones we can't access are functioning well. It's reasonably homogeneous.

4 Analysis of broken images

4.1 image_2 (which only breaks while running HPL)

The error message is:

```
/gpfs/wolf2/olcf/trn044/proj-shared/caleb_is_the_best/winterclassicaltools/
Frontier/hpl/hpl-2.3/testing/xhpl: error while loading shared libraries:
libopenblas_pthreads.so.0: cannot open shared object file:
No such file or directory
```

It cannot find libopenblas_pthreads.so.0, which is the BLAS shared object. So either Blas was not installed/doesn't exist, or the library paths are incorrect.

4.2 image_3 (which only breaks while running multinode runs)

The error is from our OSU runs. For HPL, it seems to hang and timeout:

```
Assertion failed in file src/util/mpir_pmi.c at line 473:
    got_size <= bufsize
Assertion failed in file src/mpid/ch4/netmod/ofi/ofi_init.c at line 1988:
    mapped_table[i] != FI_ADDR_NOTAVAIL
```

This is an issue with MPI, which is that it received a message larger than expected, and couldn't figure out what message was. So MPI is configured incorrectly, and whatever

protocol it is using doesn't match. However, it works on single nodes, so it is internode communication specifically (intranode communication is probably correct).

4.3 image_6 (which doesn't ever work)

We get the error:

```
Abort(2139535) on node 0 (rank 0 in comm 0): Fatal error in PMPI_Init:
Other MPI error, error stack:
MPIR_Init_thread(159).....:
MPID_Init(597).....:
MPIDI_OFI_mpi_init_hook(637):
open_fabric(1334).....:
find_provider(1431).....: OFI fi_getinfo() failure
```

This is just bad MPI. Whatever was configured doesn't work on all levels.

4.4 Image Analysis

Although probably unintended, “apptainer inspect -all image_X.sif” showed how each image was set up (there's a “runscript” field). We could use diff to see what was different between images.

For image 2: “zypper remove gcc-c++ gcc-fortran gcc13-c++ gcc13-fortran libgfortran4” is added. Not only is blas removed by libgfortran4, but c and c++ compilers are also removed. (The compilers won't affect it at runtime).

For image 3: mpich is built with “-with-device=ch4:ofi:shm” instead of “-with-device=ch4:ofi”. This explains why it works intranode but not internode, because “shm” enables shared memory, which exists on a single node but not 2.

For image 6: mpich is built with “-with-device=ch4:ofi:tcp” instead of “-with-device=ch4:ofi”. Not exactly sure what happens here, but it's likely ODO just doesn't have TCP communication enabled.

All the images use [opensuse-leap-image](#), image 1, 4, 5, 7, 8 (working images) uses version 15.6.7.49, whereas everything else (bad images 2, 3, 6) uses 15.6.7.42. The images change “dd if=/dev/zero of=/opt/foo bs=1M count=12” slightly, but this doesn't do much.

5 Additional OSU benchmark: osu_bibw

We added the osu_bibw test (OSU Bidirectional Bandwidth). The last 4 columns describe the size of the messages, and the data is in MB/s. There are many more values for other sizes, but we're showing the smallest/largest 2.

Table 7: OSU Bidirectional Bandwidth Results (MB/s)

Image	Count	1	2	2097152	4194304
image_1	393	0.3230534351	0.6510687023	6525.283232	6588.88855
image_2	394	0.3258375635	0.6542639594	6490.248629	6561.892157
image_3	1	None	None	None	None
image_4	291	0.3280068729	0.6595189003	6552.418797	6619.52323
image_5	169	0.3233136095	0.6503550296	6509.042544	6579.387101
image_6	1	None	None	None	None
image_7	179	0.3223463687	0.6469832402	6545.082961	6629.747765
image_8	247	0.3231578947	0.6508906883	6529.26996	6599.605385

We chose this benchmark since it's common that MPI messages are sent both ways simultaneously, instead of unidirectionally (which is what `osu_bw` does). You can't just take the one directional bandwidth and multiply it by 2. Since they share the same network and overhead will be less predictable with two receivers and senders, `bw` and `bibw` will test different things. Knowing how fast/big messages can be sent when nodes need to exchange data can be helpful for performance monitoring and optimization.

We can use the same graphing script to see the node-to-node bandwidth as earlier.

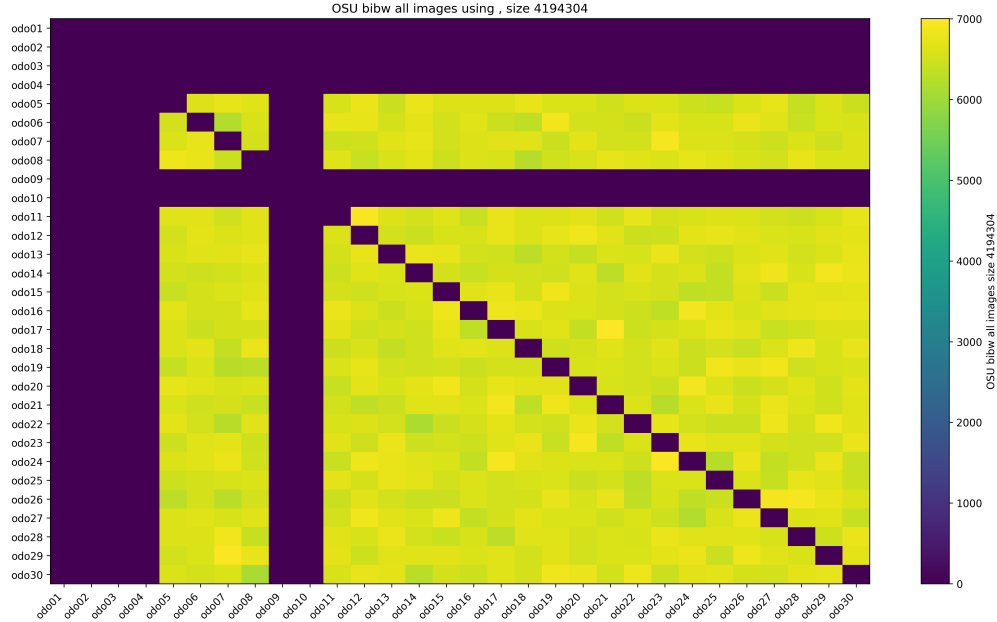


Figure 6: Bidirectional Bandwidth using size 4194304

Which exhibits roughly same homogeneity as `osu_bw`.

For automation, it was simply just adding another `srtn` line in our `run_osu.bat` batch script that was roughly identical to the `bw` and `latency` lines.

6 Automation

We write batch scripts for each test, and python scripts to automate and parse them. In our main folder, we had the `hpl`, `lammmps`, and `osu` folders as well as the images, following the setup of the instructions.

- **runAllOsu.py**
Run in a `tmux` terminal to continuously test HPL on different nodes/images. It automatically schedules the most available least tested tests. Utilizes `osu.bat`.
- **runAllHpl.py**
Run in a `tmux` terminal to continuously test OSU on different nodes/images. It automatically schedules the most available least tested tests. Utilizes `hpl_1node.bat`.
- **hpl_1node.bat**
Batch script for HPL on 1 node. Pass in image name (e.g., `image_1` to run). Automatically creates output files in the `output` folder.
- **hpl_2node.bat**
Batch script for HPL on 2 nodes. Pass in image name (e.g., `image_1` to run). Automatically creates output files in the `output` folder.
- **lammmps.bat**
Batch script for LAMMPS. Pass in image name (e.g., `image_1` to run). Automatically creates output files in the `output` folder.
- **osu.bat**
Batch script for OSU. Pass in image name (e.g., `image_1` to run). Runs bandwidth, latency, and bi-directional bandwidth tests in sequence. Automatically creates output files in the `output` folder.
- **output/collectdata.py**
Run this in the `output` folder. Collects all the data from runs, which are expected to follow the format from the `.bat` scripts. Outputs `_results.txt` files for each test, averaging results per image. Also creates charts in `output/graphs` for OSU.

We ended up running around more than 20,000 jobs (although most of them were quick `osu` benchmarks). These runs were automated primarily using 2 python scripts (`runAllHpl.py` and `runAllOSU.py`). These scripts loop indefinitely and submit batch jobs for the least tested image and node combination of HPL or OSU, respectively. It also only queues jobs that require currently available nodes. It's implemented as a priority queue, using the cartesian product of the images and nodes for all possible combinations, sorted by the number of tests (we didn't actually use python's queue, we just had a sorted list). We would then find the lowest tested available test. This way we could continuously test the nodes, while picking the most important tests: the ones we have the least information on. We would schedule a job, sleep for a bit, then figure out what the next scheduled job should be, also while keeping in mind job limits.

Queue Implementation

```
for c in OSU_counts: #[image, node1, node2], count], sorted
    if c[0][1] in nodenames and c[0][2] in nodenames: #Node availability
        run_osu(*c[0])
        c[1] += 1 #Add number of tests done
        break
OSU_counts.sort(key = lambda x:x[1])
```

To run it, we used subprocessing to schedule a job. To change the test was simple: just swap the batch script.

Subprocessing to run sbatch

```
command = [
    "sbatch",
    f"--nodelist={node1},{node2}",
    "osu.bat",
    f"{image}",
    f"{node1}",
    f"{node2}"
]

result = subprocess.run(
    command
)
```

The HPL script used a batch script for a single-node test, whereas OSU runs on two nodes. The outputs of these files were combined into one file for each workload using collectdata.py which also created graphs to help visualize the data. For each test, we wrote a different parser function in python (parseHPL, parseLAMMPS, parseOSU), which returned a dictionary of the run data.

collectdata.py example

```
HPL1_res = []
for file in HPL_1:
    res = parseHPL(os.path.join(root, "HPL_1_node", file))
    if len(res) == 0: #Failed runs just have None fields
        res = [{k:None for k in HPL_params}]
    for r in res: #Label filename
        r["run"] = file
    HPL1_res += res
HPL1_res.sort(
    key =lambda x:0 if x["Time"] is None else x["Gflops"],
    reverse = True
)
```

Once we had the data, we could reformat it nicely and do some analysis on it. Our output folder structure looks like:

output/

- `graphs/` Generated plots (OSU results)
- `HPL_1_node/` Raw outputs from single-node HPL runs
- `HPL_2_node/` Raw outputs from two-node HPL runs
- `lammps/` Raw outputs from LAMMPS runs
- `osu_bibw/` OSU bi-directional bandwidth outputs
- `osu_bw/` OSU bandwidth outputs
- `osu_latency/` OSU latency outputs
- `slurm/` Slurm job logs
- `collectdata.py` Aggregates and processes outputs
- `descriptions.txt` Run notes/descriptions
- `HPL_1_node_results.txt` Aggregated HPL (1-node)
- `HPL_2_node_results.txt` Aggregated HPL (2-node)
- `LAMMPS_results.txt` Aggregated LAMMPS results
- `OSU_bibw_results.txt` Aggregated OSU bibw
- `OSU_bw_results.txt` Aggregated OSU bw
- `OSU_latency_results.txt` Aggregated OSU latency